

medes:

- RAM utilizada
- swap usage
- inference speed
- tokens/s
- latency
- CPU utilization
- execution time
- resposta do sistema
- eventualmente memory bandwidth

O agente poderia receber:

"How much free disk space do I have?"

"Read this text file and summarize it."

Experimento 1 — Model size

Testar modelos muito pequenos:

Model A

~1B parameters

Model B

~2B parameters

Model C

~3B parameters

e observar:

Model size

↓

RAM consumption

↓

Available RAM

↓

Swap

↓

Inference performance

PCB:

Embedded AI Edge Node

com:

- STM32H755
- external SDRAM
- QSPI Flash
- microSD
- Ethernet
- USB
- OLED
- temperature sensor
- debug interface

Edge AI (Edge Artificial Intelligence) is the practice of running artificial intelligence and machine learning models directly on local hardware devices rather than on remote cloud servers.

Running an AI model on an **STM32H755ZIT3 microcontroller (MCU)** is a prime example of Edge AI, often specifically referred to as **TinyML** (Tiny Machine Learning).

The STM32H755 is a high-performance dual-core processor capable of executing optimized machine learning models (like neural networks for audio spotting, vibration analysis, or basic vision) entirely on-device without any internet or cloud connection.

Camera → "D chord" }
Mic → "D chord" } → MATCH → PASS

Start the process with a sequence of chords: C – A – G – E – D

This is like a “pin”; if correct, the OLED flashes a message and then start

Or like a guitar tutor.

And you could take it one step further: rather than training the vision model to recognize “D chord”, train it to recognize the **fretboard/finger configuration**. The audio model independently recognizes the acoustic chord. The system then learns the relationship between:

finger configuration → expected musical result

I will have: **cAiGED: a “Multimodal Edge AI Guitar Chord Verification System”**

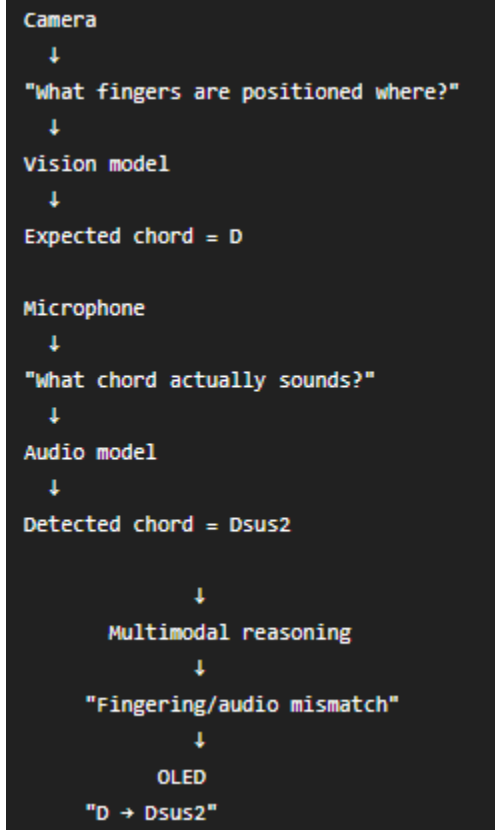
Question to answer with this project: Can an embedded AI system combine visual and acoustic information to determine whether a guitar is being played correctly?

This gives a much stronger AI story while simultaneously giving the PCB a reason to contain a camera, microphone, external memory, high-speed interfaces, OLED, Ethernet and USB.

The STM32N6 becomes justified

- Computer vision model → MIPI camera
- Audio classification model → microphone
- Multimodal fusion model → Neural-ART NPU
- STM32 firmware → acquisition, synchronization, preprocessing and decision logic
- SDRAM → image/audio buffers and model data
- microSD → datasets and recordings

- OLED → result
- USB/Ethernet → configuration/data export



Abstract

cAiGED (Multimodal Edge AI Guitar Chord Verification System) is an embedded AI platform I am developing to investigate whether a microcontroller-based system can combine visual and acoustic information to determine whether a guitar chord is being played correctly. The project is based on the **STM32N6**, selected specifically for its high-performance Edge AI capabilities, including an 800 MHz Arm Cortex-M55, 4.2 MB of on-chip RAM, and the **ST Neural-ART neural processing unit (NPU)** capable of up to 600 GOPS. The platform also provides the high-speed interfaces and computer-vision capabilities required for the proposed system, including MIPI CSI-2 camera connectivity.

The initial application will focus on the **CAGED guitar system**, using the sequence **C–A–G–E–D** as a structured learning and verification sequence. The system will observe the guitarist through a MIPI camera while simultaneously capturing the resulting guitar sound through a dedicated microphone. A computer-vision model will analyse the guitarist's hand and fretboard configuration, while an audio classification model will analyse the acoustic signal. These independent sources of information will then be combined through a multimodal inference stage to determine whether the observed finger configuration and the resulting sound are consistent with the expected chord.

Rather than limiting the system to recognising a chord directly from an image, the project will investigate the relationship between **fretboard/finger configuration and acoustic result**. The visual model will therefore aim to identify the configuration being played, while the audio model independently determines the resulting chord. Their outputs can then be combined to provide a higher-level interpretation of the performance. For example, if both modalities identify a D chord, the system can classify the attempt as a match and provide immediate feedback through the OLED display.

The project will initially operate as an interactive guitar-learning system. The CAGED sequence will provide a structured progression, with the system prompting or waiting for the expected chord and then evaluating the player's performance. A successful visual and acoustic match can trigger a **PASS** indication on the OLED before progressing to the next chord. This creates the foundation for a compact embedded guitar tutor capable of providing immediate feedback without relying on cloud-based processing.

The AI models will be developed and optimized specifically for deployment on the STM32N6. The development process will include model quantization, deployment through the **ST Edge AI** ecosystem, and validation on the target hardware. ST's current STM32N6 validation workflow supports evaluation of numerical accuracy, memory usage, NPU coverage, inference duration, NPU and Cortex-M55 cycles, external-memory traffic, bandwidth and the hardware/software workload split.

A significant part of the project will therefore be the **experimental characterization of embedded AI performance**. Different model sizes and architectures will be evaluated according to their accuracy, memory footprint and computational requirements. Measurements will include internal and external memory usage, inference latency, execution time, NPU cycles, Cortex-M55 cycles, CPU/NPU workload distribution, external-memory bandwidth, cache behaviour, CPU utilization and power consumption. The same models will also be evaluated with and without Neural-ART acceleration where appropriate, allowing the performance benefit of dedicated NPU execution to be quantified.

The custom PCB will provide the hardware infrastructure required for these experiments and for the final embedded application. The design will integrate the **STM32N6, external memory, high-speed non-volatile storage, microSD storage, MIPI camera interface, microphone/audio interface, Ethernet, USB, OLED display, environmental sensing and debug connectivity**. External memory will additionally provide an opportunity to investigate how memory placement and bandwidth affect AI inference performance, an important consideration for Neural-ART workloads. ST's Neural-ART compiler explicitly accounts for different memory pools and their access characteristics when allocating model data and activations.

The central research question of cAiGED is therefore:

Can an embedded AI system combine visual and acoustic information to determine whether a guitar is being played correctly?

Through this question, I aim to explore the practical implementation of **multimodal Edge AI on a custom embedded platform**, combining computer vision, audio classification, sensor fusion, neural-network acceleration, external memory, real-time firmware and high-speed PCB design into a single functional system.

Yes. I would now treat the abstract we just defined as the current project baseline: **STM32N6 → MIPI camera + microphone → visual/audio AI inference → multimodal comparison → chord verification**, with the CAGED sequence as the initial application. If implementation becomes too complex, we simplify peripheral features or AI scope without changing that core concept. For the thesis/project, I would **not start PCB layout immediately**. The next step should be to prove that the proposed AI pipeline is technically feasible on the STM32N6 development hardware before committing to the custom PCB.

Next step: build the proof of concept on an STM32N6 development board

ST already provides the relevant starting points. The **STM32N6570-DK** is particularly useful because it combines the STM32N6/Neural-ART NPU with MIPI camera support, external memory, Ethernet, USB, audio connectivity and SD card. ST also provides an STM32N6 audio example and vision/AI examples. ([STMicroelectronics - STM32 AI](#))

I would therefore proceed in this order:

1. Freeze the system requirements

Write a short requirements document defining:

- Expected chords: initially **C, A, G, E, D**
- Camera: MIPI CSI-2
- Audio: microphone
- Visual output: OLED
- AI: visual classification + audio classification + multimodal fusion
- Output: MATCH / MISMATCH
- Initial interaction: CAGED sequence
- Local inference: no cloud dependency
- Target MCU: STM32N657
- External memory/storage requirements
- USB/Ethernet purpose

This becomes the specification against which the PCB is designed.

2. Prove the camera AI independently

Before involving the guitar, get:

MIPI camera

↓

STM32N6

↓

Vision model



Classification

working on the development board.

For the first model, don't attempt full guitar/chord recognition. Start with something extremely simple, such as distinguishing a few controlled hand/fretboard configurations.

ST provides an official workflow for taking a quantized model, compiling it for Neural-ART and deploying it to the STM32N6. ([GitHub](#))

3. Prove the audio AI independently

Then:

Microphone



Audio preprocessing



Audio model



Chord classification

You can initially use recorded guitar samples rather than live guitar.

This is particularly useful because ST already has an STM32N6 audio example using AI-based audio event detection. ([GitHub](#))

4. Build the datasets

This is probably the first genuinely research-oriented part of the thesis.

You will need two datasets:

Visual dataset

Chord C

Chord A

Chord G

Chord E

Chord D

with many images/videos of the fretboard and hand positions.

Audio dataset

Record the corresponding chords under different conditions:

- different picking strengths;
- different strings;
- different positions;

- background noise;
- different recording distances;
- slightly different tuning.

Then split the datasets into training/validation/test sets.

5. Train the two models

You will have:

IMAGE

↓

Vision model

↓

Expected / observed fingering

AUDIO

↓

Audio model

↓

Detected chord

Do not worry about the final multimodal model yet.

First prove that each modality works independently.

6. Characterize the models on the STM32N6

This is where your earlier benchmarking plan comes in.

For each model, measure:

- model size;
- accuracy;
- RAM usage;
- external-memory usage;
- inference latency;
- NPU cycles;
- Cortex-M55 cycles;
- CPU/NPU workload;
- memory bandwidth;
- external-memory traffic;
- power consumption.

ST's own N6 tooling provides profiling and validation functionality for exactly these kinds of measurements. ([dev.st](#))

Also perform the particularly useful:

Neural-ART vs Cortex-M55-only comparison.

That gives you a quantitative justification for choosing the STM32N6 rather than a conventional MCU. ST explicitly provides separate NPU and Cortex-M55 validation projects. ([dev.st](#))

7. Only then implement multimodal fusion

Once you have:

Camera → "D" 92%

Mic → "D" 87%

implement:

Vision evidence

+

Audio evidence

↓

Multimodal fusion

↓

D / confidence

↓

MATCH

This is the point where cAiGED actually becomes the **multimodal AI system** described in your abstract.

8. Implement the CAGED tutor

Then introduce the sequence:

C → A → G → E → D

For example:

EXPECTED: C

Camera → C 94%

Audio → C 91%

FUSION → C 97%

OLED:

✓ C CHORD

PASS

Next: A

Now you have a complete proof of concept.

Then design the custom PCB

Only after steps 1–8 should we freeze the PCB architecture.

At that point we will know what the actual requirements are for:

- STM32N657;
- camera interface;
- microphone/audio interface;
- RAM;
- Flash;
- microSD;
- USB;
- Ethernet;
- OLED;
- power rails;
- clocking;
- debug;
- connectors;
- high-speed routing.

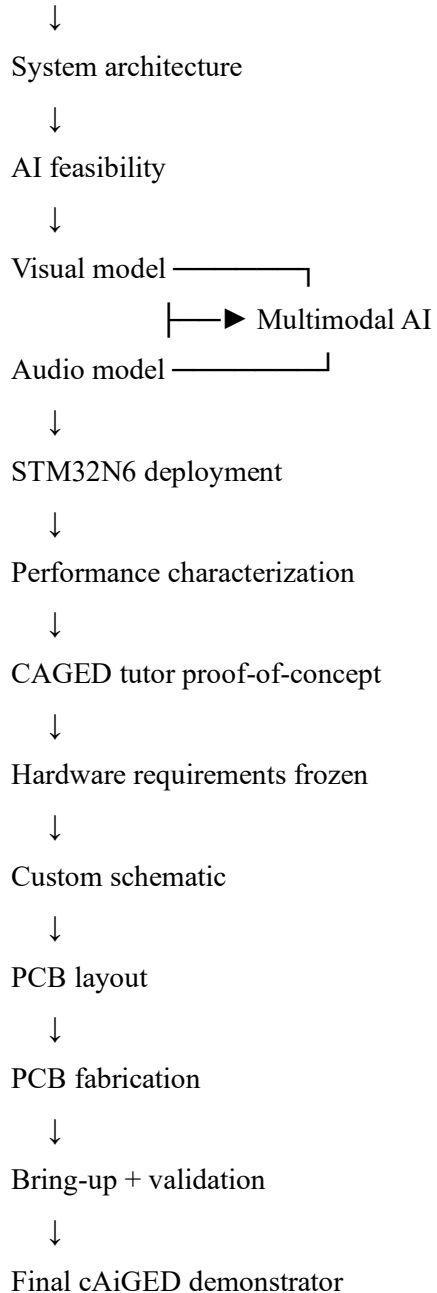
This is particularly important because **AN5967 is specifically intended to guide STM32N6 custom hardware development**, covering power supply, package selection, clock management, reset, boot modes, debug and reference schematics. I would make AN5967 one of the primary documents for the schematic phase. ([STMicroelectronics](#))

And there is one very important point from ST's current documentation: the **NUCLEO-N657X0-Q has no external RAM**, while ST recommends the STM32N6570-DK for the full AI evaluation workflow when external RAM is required. ([ST Edge AI](#))

So, if your goal is ultimately a serious custom PCB with external memory, I would use the **STM32N6570-DK as the initial AI reference platform**, then derive your custom hardware architecture from it.

The overall thesis progression should therefore be:

Requirements



So the immediate next task I recommend is not PCB design yet. It is to create the cAiGED System Requirements + Architecture document and then set up the STM32N6 development environment and prove one very small vision inference on the development board. That gives us a technically grounded starting point instead of designing a PCB around assumptions.

cAiGED — Project Baseline & Engineering Handover Report

This document is intended to be used as a complete context handover for a new ChatGPT conversation. It consolidates the project concept, current hardware architecture, reference-design strategy, AI concept, development methodology, known decisions, assumptions, and unresolved engineering questions.

The report distinguishes between established project decisions, information derived from official external documentation, and areas that are still open. Where a detail has not been formally established, it is explicitly marked as such rather than being invented.

1. Project identity

Project name: **cAiGED**

Project category: **Embedded Edge-AI / multimodal embedded electronics / guitar education**

Primary engineering domains:

- Embedded hardware
- STM32
- Edge AI / TinyML
- Computer vision
- Audio processing
- Multimodal machine learning
- PCB design
- High-speed digital design
- External memory
- Camera interfaces
- Audio acquisition
- Embedded firmware
- Real-time systems
- Human-machine interaction

The project is intended to result in a custom embedded hardware platform capable of performing AI-assisted guitar chord verification locally, rather than relying on a PC or cloud service.

The system combines two fundamentally different observations of the guitarist:

1. **Visual observation** — what the guitarist's fingers are physically doing on the fretboard.
2. **Acoustic observation** — what the guitar actually sounds like.

These two observations are intended to be processed independently and subsequently combined.

2. Core concept

The central concept is a multimodal guitar-learning system that can determine whether a guitarist has correctly performed a target chord.

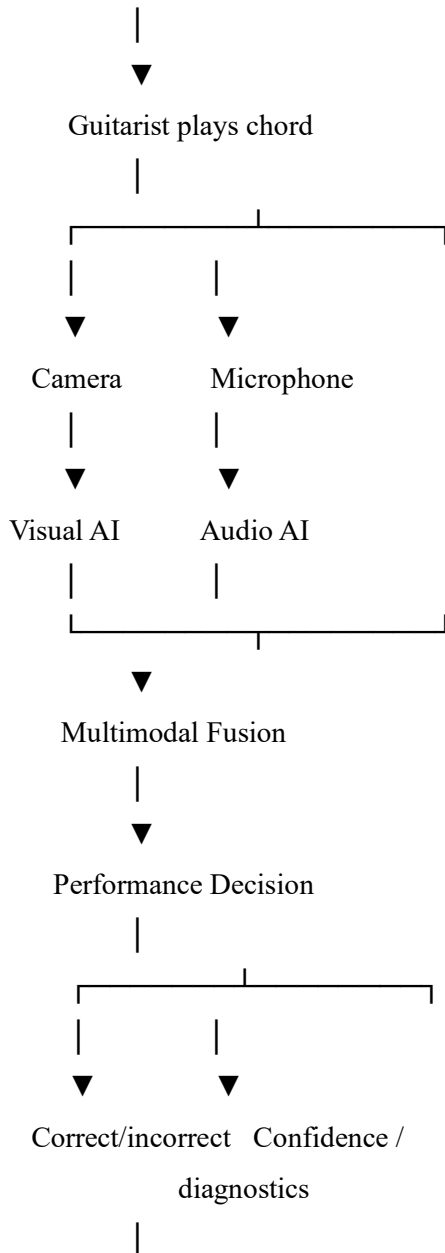
The initial musical scope is based on the **CAGED system**:

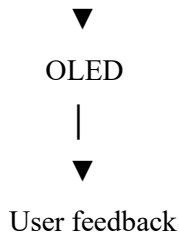
C → **A** → **G** → **E** → **D**

The system is therefore initially intended to work with a constrained set of guitar chord shapes rather than attempting to recognize every possible guitar chord or arbitrary musical performance.

The basic interaction is:

User selects / is presented with target chord





The important conceptual distinction is that cAiGED is not merely a camera-based chord classifier.

The system should ideally establish a relationship between:

physical finger configuration → expected chord

and independently evaluate:

acoustic output → actual sounding chord.

The two observations can then be compared.

3. Why multimodal sensing is important

A guitar chord can fail for several different reasons.

A guitarist may:

- place one finger on the wrong fret;
- place a finger on the correct fret but insufficiently press the string;
- accidentally mute another string;
- play an incorrect string;
- have the correct finger configuration but produce an incorrect sound;
- have an apparently incorrect finger configuration that nevertheless produces the intended chord;
- produce a noisy or ambiguous acoustic result.

A purely visual system cannot reliably determine everything that happened acoustically.

A purely acoustic system also cannot necessarily explain why the chord was wrong.

The multimodal architecture therefore potentially enables two complementary forms of information.

Visual AI can answer questions such as:

"Are the fingers positioned according to the expected shape?"

Audio AI can answer:

"Did the resulting sound correspond to the expected chord?"

The fusion system can potentially distinguish between these cases.

For example:

Visual: correct

Audio: correct

→ likely successful chord

Visual: incorrect

Audio: incorrect

→ likely incorrect fingering

Visual: correct

Audio: incorrect

→ possible muting / articulation / incomplete fretting / string-selection issue

Visual: uncertain

Audio: correct

→ acoustically successful, visually ambiguous

The exact interpretation of these combinations is not yet frozen and should ultimately be established experimentally.

4. Initial functional objective

The first version should be deliberately constrained.

The project should not initially attempt:

- arbitrary chord recognition;
- complete guitar transcription;
- full sheet-music interpretation;
- polyphonic transcription of arbitrary guitar playing;
- general-purpose guitarist assessment;
- complete automated guitar tutoring.

The initial objective is much more controlled:

Recognize and verify a defined set of guitar chord shapes, beginning with the CAGED-related chord sequence, using visual and acoustic information on an embedded Edge-AI platform.

This constraint is valuable because it dramatically reduces the classification problem and makes the project experimentally tractable.

5. Target hardware architecture

The current MCU baseline is:

STM32N657X0

More specifically, the currently relevant production ordering is:

STM32N657X0H3Q

ST lists the STM32N657X0H3Q as an active, volume-production device in a **VFBGA-264, 14 × 14 mm, 0.8 mm pitch** package. ([STMicroelectronics](#))

This is important because the project is not based on a conventional low-end STM32.

The STM32N657X0 is explicitly designed for high-performance embedded processing and Edge AI.

ST specifies:

- Arm Cortex-M55
- up to 800 MHz CPU frequency
- Arm Helium/M-Profile Vector Extension
- 32 KB instruction cache
- 32 KB data cache
- 128 KB data TCM
- 64 KB instruction TCM
- 4.2 MB contiguous SRAM
- external memory controller
- XSPI interfaces
- Neural-ART accelerator
- up to 600 GOPS
- NeoChrom graphics acceleration
- JPEG hardware codec
- H.264 encoding
- MIPI CSI-2 camera interface
- image signal processor
- Ethernet
- USB
- extensive peripheral interfaces

ST's current product documentation confirms the 800 MHz Cortex-M55, 4.2 MB SRAM and 600 GOPS Neural-ART architecture. ([STMicroelectronics](#))

6. Why the STM32N657X0 was selected

The MCU selection is driven primarily by the Edge-AI requirement.

A conventional STM32 could handle:

- GPIO
- display
- basic audio acquisition
- USB
- control logic

but the project requires substantially more computational capability because it potentially needs to perform:

- image acquisition;
- image preprocessing;
- computer-vision inference;
- audio preprocessing;
- audio inference;
- multimodal processing;
- memory movement;
- real-time control;
- user interface management.

The Neural-ART accelerator is particularly important because it is specifically intended to accelerate neural-network workloads.

ST specifies the STM32N6 Neural-ART accelerator at up to 1 GHz and 600 GOPS, with dedicated hardware for DNN inference and streaming/data-flow optimization. ([STMicroelectronics](#))

Therefore, the MCU is not simply the "controller" of cAiGED.

It is intended to be the computational core of the entire embedded AI platform.

7. STM32N657X0 memory architecture

The MCU contains substantial internal memory, but the project should not assume that all application data and AI models can simply reside inside the MCU.

ST specifies:

- 4.2 MB contiguous SRAM;
- TCM memory for critical real-time data/code;
- flexible external memory controller;
- external SRAM/PSRAM/SDRAM support;
- XSPI interfaces for external serial memory. ([STMicroelectronics](#))

Consequently, external memory is an important part of the cAiGED hardware architecture.

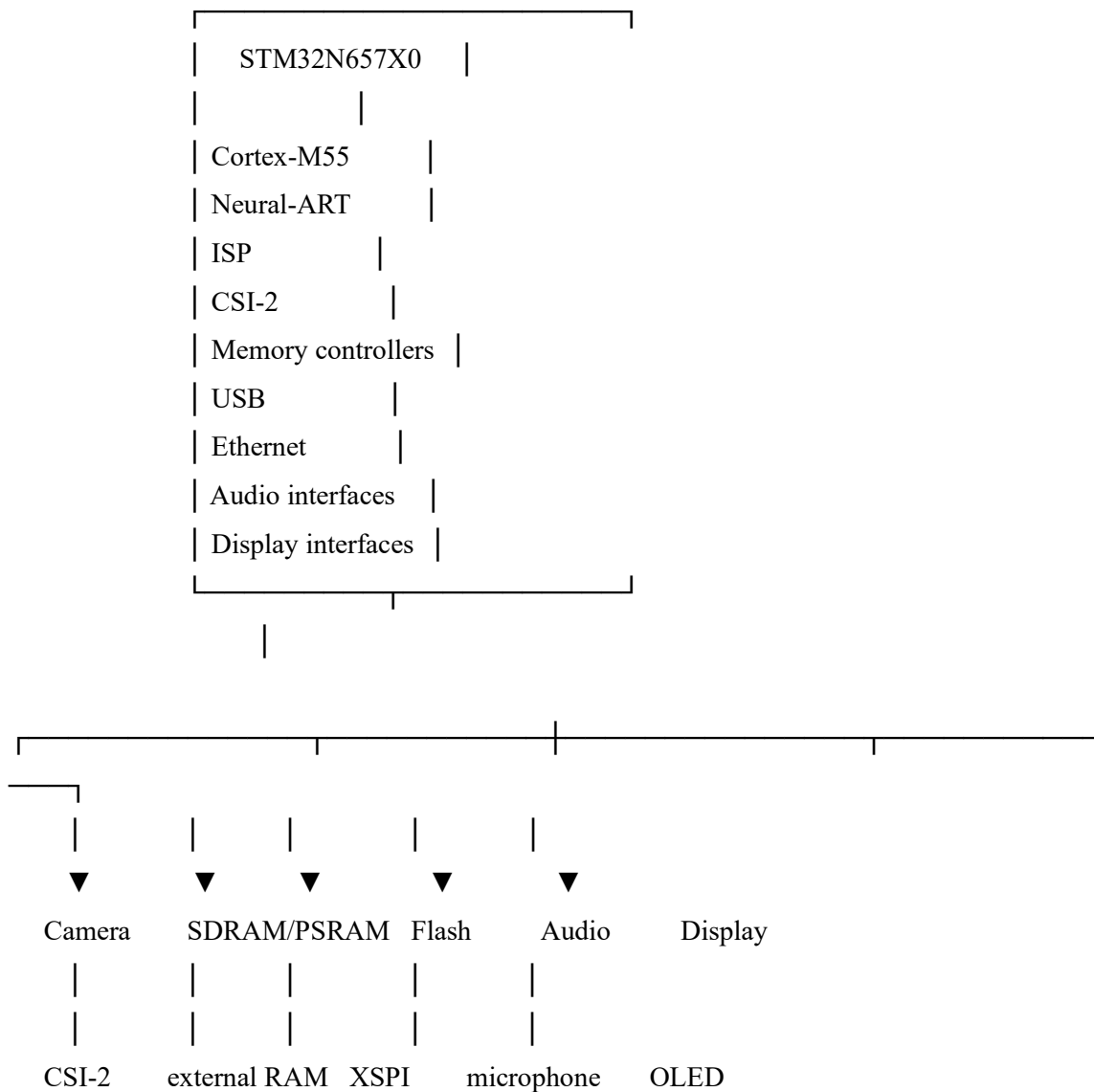
The project currently expects external memory for things such as:

- frame buffers;
- neural-network working memory;
- model data;
- audio buffers;
- graphics buffers;
- application data.

The exact memory partitioning is still an engineering task.

8. Current peripheral architecture

The working hardware concept includes the following major functional blocks:



Additional interfaces under consideration/currently included in the baseline concept:

- USB
- Gigabit Ethernet
- microSD
- debug/programming interface

The exact implementation of each interface remains subject to the final system requirements.

9. Camera subsystem

The camera is a fundamental part of cAiGED.

Its purpose is not merely to capture a general image of the guitarist.

The visual model needs useful information about the guitar's fretboard and finger positions.

Therefore, camera placement, lens characteristics, field of view, lighting and mechanical geometry will be important to system performance.

The STM32N6 family is particularly suitable because ST provides:

- MIPI CSI-2;
- a dedicated image signal processor;
- image processing functions;
- hardware-assisted image pipelines.

ST specifies a 2-lane CSI-2 camera interface and an ISP with functions including cropping, downscaling, region-of-interest processing, demosaicing, gamma and pixel packing.

([STMicroelectronics](#))

ST also provides official STM32N6 camera examples using the NUCLEO-N657X0-Q and camera modules such as the IMX335. ([GitHub](#))

This is useful for the project because the camera subsystem can be experimentally validated on ST's development platform before being incorporated into the custom PCB.

10. Camera AI concept

The exact visual neural-network architecture is not frozen.

The expected processing chain is approximately:

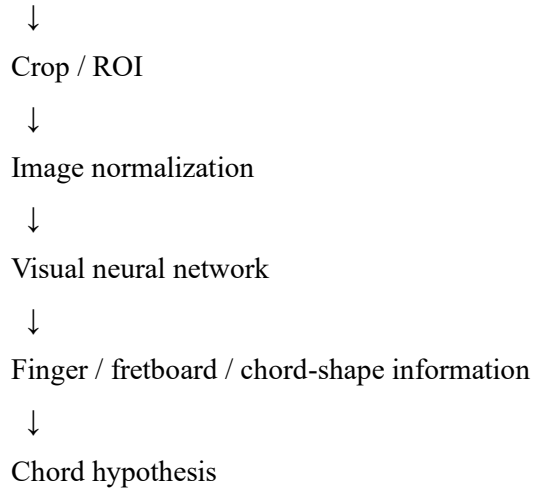
Camera

↓

Frame acquisition

↓

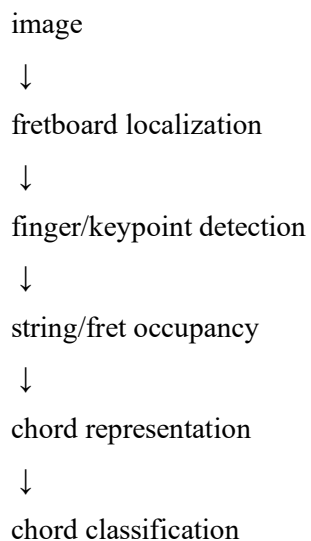
ISP preprocessing



An important architectural question is whether the visual network should directly classify:

image $\rightarrow$ C / A / G / E / D

or instead extract a more interpretable representation such as:



The second architecture may potentially provide more useful diagnostic information, but this is not currently a final decision.

It must be evaluated against:

- dataset size;
- inference complexity;
- accuracy;
- latency;
- interpretability;
- Neural-ART compatibility;
- development time.

11. Audio subsystem

The second major sensing modality is audio.

The microphone is intended to capture the guitar's acoustic output while the guitarist performs the chord.

The audio pipeline is expected to look approximately like:

Guitar



Microphone



Audio acquisition



Digital/analog conversion



Preprocessing



Time-frequency representation



Audio neural network



Chord classification

The exact microphone technology is not yet frozen.

Possible architectures could involve:

- digital MEMS microphone;
- analog microphone + ADC;
- audio codec;
- I²S/SAI-based acquisition.

The final choice needs to be based on:

- audio quality;
- noise rejection;
- required bandwidth;
- sampling frequency;
- PCB complexity;
- STM32N6 interface availability;

- latency;
- power consumption.

This should not be treated as decided until the audio requirements are defined.

12. Audio AI concept

The acoustic model should determine the musical information relevant to chord verification.

The initial problem is intentionally constrained.

Rather than requiring arbitrary guitar transcription, the system can initially classify among the target chord set.

For example:

Audio input

↓

windowing

↓

spectral preprocessing

↓

feature representation

↓

neural network

↓

{C, A, G, E, D, other/uncertain}

Potential representations may include:

- waveform;
- STFT;
- spectrogram;
- mel-spectrogram;
- chroma-like features.

The exact representation and model are open.

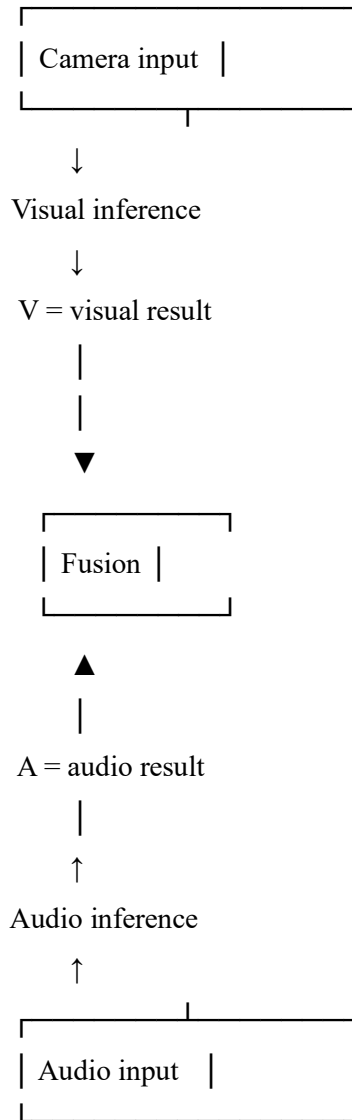
The model must ultimately be selected based on experimental results rather than assumed in advance.

13. Multimodal fusion

This is one of the most important aspects of cAiGED.

The project should maintain separate visual and audio processing paths before fusion.

Conceptually:



The fusion layer could theoretically use:

- hard decision fusion;
- confidence-weighted fusion;
- feature-level fusion;
- late neural fusion;
- rule-based fusion;
- probabilistic fusion.

No final fusion method has been established yet.

The architecture should therefore be designed so that the two models can initially operate independently.

This will also make development and debugging substantially easier.

14. Feedback mechanism

The current concept includes an **OLED display** as the local user interface.

The display is intended to communicate things such as:

- current target chord;
- detected chord;
- correct/incorrect result;
- progression through the exercise;
- potentially confidence or diagnostic information.

The exact UI is not frozen.

The important architectural principle is that cAiGED should provide immediate local feedback without requiring a computer.

15. CAGED progression

The first intended learning sequence is:

C

↓

A

↓

G

↓

E

↓

D

The system can therefore operate as an exercise loop:

Select target

↓

Show target chord

↓

User positions fingers

↓

Camera observes

↓

User plays



Microphone observes



AI inference



Fusion



Pass / fail



Feedback



Next chord

The exact progression rules remain to be defined.

For example, it is not yet formally established:

- how many successful attempts are required;
- whether multiple consecutive successes are required;
- how uncertainty is handled;
- whether a failed attempt gives a second chance;
- whether the system should identify specific fingering errors;
- how the exercise difficulty should evolve.

16. Current hardware reference strategy

This is the most important PCB-development decision currently established.

The custom cAiGED PCB should **not** be designed from an empty schematic.

The hardware baseline is:

STMicroelectronics NUCLEO-N657X0-Q

with motherboard/reference design:

MB1940

ST provides the MB1940 board design project files, manufacturing files, BOM and schematic as official resources. The current ST page lists the MB1940 design files and schematic as version 1.0, updated October 18, 2024. ([STMicroelectronics](#))

The official MB1940 schematic is a 15-sheet design.

Its sheets cover:

1. Project overview
2. Top
3. STM32 microcontroller I/Os
4. STM32 microcontroller power
5. OctoSPI Flash
6. Connectors
7. Ethernet
8. USB
9. Power
10. Arduino extension connectors
11. Camera
12. Peripherals
13. STLINK_V3EC
14. External debug interface
15. Mechanical

This structure is explicitly visible in ST's official schematic pack. ([STMicroelectronics](#))

17. Meaning of the MB1940 baseline

MB1940 is a **reference design**, not the final cAiGED design.

The intended development process is:

Official ST MB1940



Study architecture



Understand every circuit



Port relevant circuitry to KiCad



Remove Nucleo-specific circuitry



Adapt memory / power / interfaces



Add cAiGED-specific peripherals



Perform electrical design review



PCB layout



Prototype



Bring-up



Validation

The objective is not to blindly copy the board.

Each circuit should be classified as:

- required;
- useful;
- optional;
- Nucleo-specific;
- unnecessary;
- requiring redesign.

18. Nucleo-specific circuitry

The NUCLEO-N657X0-Q contains several features intended specifically for development-board use.

ST lists features including:

- onboard ST-LINK;
- user LEDs;
- push-buttons;
- ST morpho headers;
- Arduino connectors;
- camera connector;
- MIPI connector;
- USB;
- Ethernet;
- CAN FD header;
- memory connector. ([STMicroelectronics](#))

Not all of these should automatically appear in cAiGED.

For example, the following are likely development-board-specific:

- ST-LINK subsystem;
- Arduino expansion connectors;
- Nucleo-specific headers;
- development-board user buttons;
- some board-specific connector infrastructure.

However, this does not mean that all such circuits should simply be deleted.

The custom board still needs an appropriate:

- debug interface;
- reset mechanism;
- boot/configuration mechanism;
- programming method;
- test access.

The exact solution remains to be designed.

19. Why using MB1940 is valuable

The STM32N657X0 is a significantly more complicated MCU than the microcontrollers used in many previous PCB projects.

The difficult parts include:

- BGA package;
- high pin count;
- multiple power domains;
- external memory;
- high-speed interfaces;
- camera interface;
- Ethernet;
- USB;
- high-performance clocks;
- AI memory bandwidth;
- high-speed PCB routing.

Using an official ST board reduces unnecessary risk.

Instead of independently deriving the complete STM32N6 power and boot architecture, the project can begin from a design that has already been produced by ST.

This is especially important for:

- power sequencing;
- decoupling;
- memory connectivity;
- boot configuration;
- clocking;
- high-speed interfaces;
- pin assignments.

It does not eliminate the need for engineering verification, but it gives us a strong reference implementation.

20. KiCad migration

The user's normal PCB design environment is **KiCad/Altium**, and cAiGED's custom design is intended to be developed in KiCad.

The MB1940 source design is provided by ST as a board design project, while the official schematic is in Altium format. ST explicitly makes the board design project files available for the NUCLEO-N657X0-Q. ([STMicroelectronics](#))

The migration therefore becomes an engineering task in itself.

The correct approach is not:

import everything → trust the importer → start routing.

Instead:

Altium reference

↓

Import / reconstruct

↓

Compare against official PDF

↓

Compare against datasheets

↓

Verify symbols

↓

Verify footprints

↓

Verify net names

↓

Verify component values



Verify power domains



Verify pin assignments



Verify design rules



Only then continue development

Every imported section should be reviewed.

21. Existing PCB-design experience relevant to cAiGED

Previous engineering material available in the project context demonstrates experience with:

- STM32;
- USB;
- Ethernet;
- RF;
- controlled impedance;
- differential pairs;
- four-layer PCB architecture;
- power distribution;
- decoupling;
- ESD;
- signal-integrity considerations.

For example, the earlier STM32-RF-USB-Gateway design uses:

- STM32L432;
- USB Full-Speed;
- nRF24L01P;
- SMA;
- RF matching network;
- SWD;
- 3.3 V LDO;
- USB ESD protection.

The schematic explicitly documents 50 Ω RF routing, 90 Ω USB differential routing, local MCU decoupling and RF layout constraints.

The associated design material also describes the design methodology used for:

- four-layer stackups;
- controlled impedance;
- differential-pair routing;
- RF guard rings;
- ground planes;
- via placement;
- high-speed routing separation;
- DRC;
- Gerber generation;
- Gerber inspection.

That previous experience is relevant because cAiGED is essentially a much more demanding version of the same class of embedded PCB problem.

22. Previous Ethernet experience

Another previous design available in the project material is a USB 3.2 Gen 1 to Gigabit Ethernet adapter using:

- LAN7800;
- external Ethernet magnetics/RJ45;
- USB 3.x;
- multiple regulated power rails;
- high-speed differential interfaces;
- controlled power distribution.

The schematic shows a LAN7800-based architecture with 3.3 V, 2.5 V and 1.2 V domains, external 25 MHz clocking, Ethernet transformer/RJ45 and USB 3.x connections.

This experience is relevant to cAiGED's Ethernet subsystem, although the cAiGED Ethernet implementation must follow the STM32N6/MB1940 architecture rather than simply reusing the old LAN7800 architecture.

23. Ethernet

Gigabit Ethernet is part of the current hardware concept.

The NUCLEO-N657X0-Q provides Ethernet through an external PHY using RMII. ([GitHub](#))

The exact role of Ethernet in cAiGED is not yet formally frozen.

Possible uses include:

- development/debugging;
- firmware updates;
- dataset transfer;
- remote diagnostics;
- test automation;
- network connectivity;
- future cloud/PC integration.

It should therefore be retained in the architecture until the requirements are explicitly reduced.

However, the final board should not automatically copy every Ethernet feature from the Nucleo.

24. USB

USB is also part of the current system architecture.

The precise role is not yet frozen.

Potential uses include:

- firmware programming;
- debugging;
- PC communication;
- data transfer;
- development;
- dataset extraction;
- device configuration.

The exact USB topology, connector and speed requirements remain open.

This should be defined from the intended product workflow before finalizing the connector and protection circuitry.

25. microSD

microSD is part of the current system concept.

Its likely role is storage for:

- datasets;
- audio recordings;
- captured images;
- diagnostic data;

- model files;
- configuration;
- firmware or update assets.

However, it should not be assumed that the AI runtime will execute directly from microSD.

The high-performance inference path should use the appropriate internal/external memory architecture.

microSD should be considered removable storage rather than the primary high-bandwidth AI memory.

26. External Flash

The STM32N6 architecture supports external serial memory through XSPI.

The MB1940 includes an OctoSPI Flash section. The official schematic identifies this as a dedicated section of the board design. ([STMicroelectronics](#))

External Flash is potentially required for:

- boot images;
- firmware;
- AI model storage;
- assets;
- application data.

The exact memory density and device selection remain to be finalized.

27. External RAM

External RAM is particularly important for cAiGED because image processing and neural inference can require large temporary buffers.

The MCU's internal 4.2 MB SRAM is substantial, but external memory provides additional capacity and potentially enables:

- larger frame buffers;
- multiple image buffers;
- audio buffers;
- neural-network working memory;
- graphics buffers.

The STM32N6 supports external SDRAM/PSRAM through its external memory interfaces. ([STMicroelectronics](#))

The exact memory technology and device remain open.

A proper memory-budget exercise is required before final selection.

28. AI memory architecture

The AI architecture should distinguish between different memory classes.

Conceptually:

Internal TCM

|
 |— real-time critical code
 |— latency-sensitive data

Internal SRAM

- |
- |— application buffers
- |— control structures
- |— smaller AI data

External RAM

- |
- |— camera frame buffers
- |— audio buffers
- |— large tensors
- |— model working memory

External Flash

- |
- |— firmware
- |— AI model
- |— assets

microSD

- |
- |— datasets
- |— recordings
- |— logs

└─ removable data

The final mapping should be derived from actual model and buffer sizes.

29. AI deployment strategy

ST provides a software ecosystem around STM32N6 Edge AI.

ST's STM32N6 model-deployment examples use **ST Edge AI Core** to generate network C code from formats such as TensorFlow Lite and ONNX and use STM32CubeIDE for building the embedded application. ([GitHub](#))

Therefore, the expected AI development flow is approximately:

Dataset



Model development



Training



Validation



Quantization / optimization



ST Edge AI tooling



Neural-ART compatible deployment



Embedded inference



Performance measurement

The exact AI framework is not yet frozen.

30. Model-development philosophy

The project should not start by selecting an unnecessarily large neural network.

The intended philosophy should be:

use the smallest model that provides the required classification performance while meeting the embedded real-time constraints.

Important metrics should include:

- accuracy;
- precision;
- recall;
- confusion matrix;
- false-positive rate;
- false-negative rate;
- inference latency;
- RAM consumption;
- Flash/model size;
- power consumption.

For multimodal inference, additional metrics are needed:

- visual-only accuracy;
- audio-only accuracy;
- fused accuracy;
- failure cases;
- disagreement rate between modalities.

31. Dataset requirements

The dataset is likely to be one of the most important determinants of project success.

The visual dataset must capture variation in:

- guitarist;
- hand size;
- skin tone;
- guitar type;
- fretboard appearance;
- lighting;
- camera angle;
- finger position;
- chord quality;
- background;
- clothing;
- playing technique.

The acoustic dataset must capture variation in:

- guitar;

- strings;
- tuning;
- microphone position;
- room;
- background noise;
- playing strength;
- pick/finger style;
- chord voicing;
- muting;
- articulation.

A serious concern is dataset leakage.

For example, the same physical recording session should not simply be split randomly between training and validation if the system can learn session-specific characteristics.

The evaluation dataset should represent genuinely unseen conditions.

32. Synchronization

The multimodal architecture requires temporal synchronization.

The camera observes:

finger configuration

while the microphone observes:

acoustic event.

These events are not necessarily simultaneous.

There is potentially a sequence such as:

finger movement

↓

finger reaches position

↓

settling time

↓

strum

↓

sound begins

↓

sound evolves

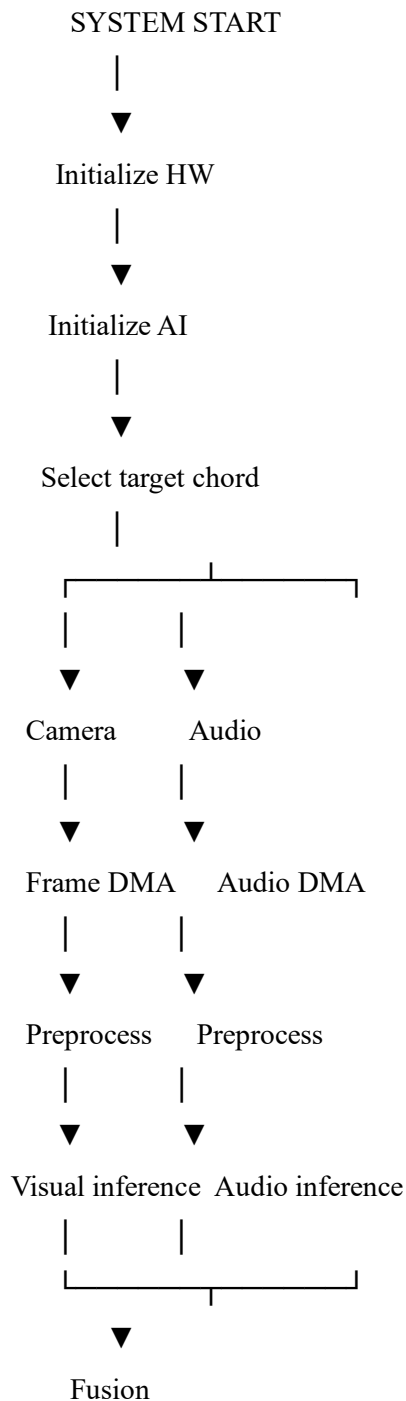
Therefore, the system should eventually establish an observation window around the playing event.

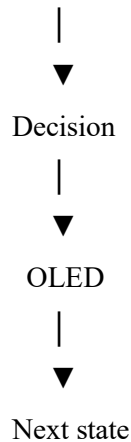
The exact timing architecture is still open.

This is a significant software/AI research problem, not merely a hardware detail.

33. Embedded real-time pipeline

The eventual embedded pipeline may look approximately like:





DMA should be used heavily where appropriate so that CPU/NPU execution is not unnecessarily blocked by data acquisition.

34. FreeRTOS / concurrency

The project is likely to benefit from a multitasking architecture.

The exact RTOS choice is not frozen, but FreeRTOS is particularly relevant because the user's previous embedded development includes FreeRTOS training and STM32 development.

Potential task separation:

Camera acquisition task

Audio acquisition task

Visual preprocessing task

Audio preprocessing task

AI inference task

Fusion task

UI task

Storage task

Communication task

System monitoring task

However, this should not be implemented simply because "RTOS is available."

The actual scheduling architecture should be derived from:

- timing constraints;
- DMA;
- ISR load;
- NPU execution;
- memory bandwidth;

- latency requirements.

35. Hardware/software boundary

One important project principle is that the hardware must support the AI architecture rather than being designed independently of it.

For example:

AI model requirements



tensor sizes



buffer sizes



memory bandwidth



external RAM selection



PCB architecture

Likewise:

camera model



resolution / FPS



CSI-2 bandwidth



memory bandwidth



frame-buffer requirements



external RAM

And:

audio model



sampling rate



buffer size



audio interface



memory/DMA architecture

Therefore, the hardware design should not be finalized independently of the AI/software requirements.

36. Power architecture

The STM32N6 system contains multiple power requirements.

The exact final cAiGED power tree has not yet been frozen.

The MB1940 reference design should be treated as the starting point for understanding:

- MCU power domains;
- core supply;
- I/O supplies;
- memory supply;
- Ethernet supply;
- USB supply;
- external peripheral supplies;
- decoupling;
- power sequencing.

The custom board should reproduce only the required power architecture, after verifying every rail against the final peripheral selection.

37. Clock architecture

Clocking is another area where the MB1940 reference should be studied carefully.

The STM32N6 contains high-performance processing and multiple high-speed peripherals, so clock architecture should not be simplified prematurely.

Relevant clocks may include:

- MCU system clock;
- external crystal/oscillator;
- memory clocks;
- camera clocking;
- Ethernet clock;

- USB clock;
- audio clock.

The final oscillator selection and clock-tree implementation need to be validated against:

- STM32N6 datasheet;
 - reference manual;
 - MB1940 schematic;
 - selected peripherals;
 - final PCB constraints.
-

38. PCB architecture

The final PCB will be significantly more demanding than the user's previous STM32/USB/RF boards.

Expected critical PCB areas include:

- BGA escape;
- high-speed memory;
- XSPI;
- MIPI CSI-2;
- Ethernet;
- USB;
- power distribution;
- clock routing;
- audio;
- display interface.

The PCB should therefore be designed around the high-speed constraints rather than treating them as a final routing step.

39. Layer stack-up

The final stack-up is currently open.

However, because the design contains:

- MIPI;
- high-speed memory;
- Ethernet;
- USB;
- high-speed clocks;

- BGA;
- high-performance MCU;

the stack-up must be engineered deliberately.

The final stack-up should define:

- number of copper layers;
- copper thickness;
- dielectric thickness;
- dielectric constants;
- controlled-impedance layers;
- reference planes;
- power planes;
- via structures.

Impedance calculations should be based on the actual fabricator stack-up.

The previous PCB project methodology already established the importance of calculating impedance from the manufacturer's stack-up rather than selecting arbitrary track widths.

40. BGA considerations

The STM32N657X0H3Q uses a 264-ball VFBGA package with 0.8 mm pitch.
([STMicroelectronics](#))

This introduces significantly more challenging PCB constraints than an LQFP MCU.

Important issues include:

- fanout;
- via diameter;
- via pad size;
- escape routing;
- solder-mask rules;
- fabrication capability;
- power/ground via density;
- breakout-layer strategy;
- impedance control;
- return-current paths.

The final PCB manufacturer must therefore be selected before freezing the stack-up and BGA escape strategy.

41. High-speed interface philosophy

The project should treat each high-speed interface according to its actual electrical requirements.

Potentially critical interfaces include:

- MIPI CSI-2;
- XSPI;
- Ethernet;
- USB;
- high-speed clocks.

The design methodology should include:

- impedance control;
- reference-plane continuity;
- differential-pair constraints where applicable;
- length/skew constraints where required;
- via minimization;
- return-path control;
- crosstalk analysis;
- connector transitions;
- power integrity.

The previous PCB design work already emphasizes uninterrupted reference planes, controlled impedance and minimizing coupling between unrelated high-speed interfaces.

42. Debugging strategy

The custom board must retain a robust debugging strategy.

At minimum, development access should allow:

- programming;
- reset;
- debugging;
- firmware recovery;
- serial/log output where appropriate;
- hardware test points.

The Nucleo board integrates ST-LINK, but cAiGED is not expected to simply reproduce the entire ST-LINK subsystem.

The exact production/debug connector architecture remains open.

43. Manufacturing/test philosophy

Because the project is a custom embedded hardware system, testability should be considered during schematic and PCB design.

Potential test provisions include:

- test points;
- power-rail test pads;
- SWD;
- UART;
- USB;
- boundary-accessible signals;
- memory test points where useful;
- boot configuration access;
- reset access.

The user's professional experience includes developing dedicated PCB production-test systems and C# test applications, which is directly relevant to the eventual cAiGED manufacturing/test strategy.

44. Development stages

The project should be treated as several interconnected engineering stages.

Stage 1 — System requirements

Define:

- target chords;
- camera requirements;
- audio requirements;
- inference latency;
- display requirements;
- storage;
- connectivity;
- power;
- physical form factor.

Stage 2 — AI feasibility

Develop independent:

- visual model;
- audio model.

Measure:

- accuracy;
- size;
- latency;
- resource requirements.

Stage 3 — Multimodal algorithm

Implement:

- synchronization;
- confidence handling;
- fusion;
- decision logic.

Stage 4 — Hardware reference study

Study MB1940:

- MCU;
- power;
- memory;
- Ethernet;
- USB;
- camera;
- debug.

Stage 5 — KiCad architecture

Create:

- project structure;
- symbols;
- footprints;
- schematic hierarchy;
- net naming;
- design rules.

Stage 6 — Custom schematic

Replace Nucleo-specific circuitry with cAiGED-specific architecture.

Stage 7 — PCB

Perform:

- stack-up;
- BGA escape;
- placement;

- power;
- high-speed routing;
- impedance;
- DRC.

Stage 8 — Prototype

Perform:

- power bring-up;
- MCU programming;
- memory validation;
- camera validation;
- Ethernet/USB validation;
- audio validation;
- display validation.

Stage 9 — AI deployment

Port the actual models.

Stage 10 — System validation

Measure:

- multimodal accuracy;
- latency;
- power;
- thermal behaviour;
- robustness.

45. Prototype philosophy

The project should avoid making the first custom PCB responsible for proving every unknown simultaneously.

The safer sequence is:

NUCLEO-N657X0-Q

↓

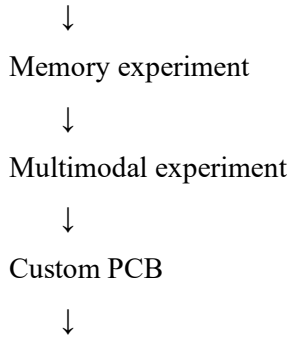
Camera experiment

↓

Audio experiment

↓

AI inference experiment



Custom hardware validation

This is particularly important because an AI algorithm failure and a PCB failure can otherwise become difficult to distinguish.

46. Reference development hardware

The **NUCLEO-N657X0-Q** is therefore the primary development platform/reference platform.

ST confirms that this board includes the STM32N6 device and provides interfaces including:

- Ethernet;
- USB;
- camera;
- memory;
- debug;
- expansion interfaces. ([STMicroelectronics](#))

It is therefore suitable for validating substantial parts of the cAiGED concept before the custom PCB exists.

ST also provides camera AI examples for the NUCLEO-N657X0-Q. ([GitHub](#))

47. Important distinction: NUCLEO vs cAiGED

This distinction should always be preserved in future conversations.

NUCLEO-N657X0-Q:

Development board.

Purpose:

- experimentation;
- debugging;
- evaluation;
- reference hardware.

MB1940:

Official ST board design.

Purpose:

- reference implementation of the STM32N657X0 hardware.

cAiGED PCB:

Custom product/research hardware.

Purpose:

- integrate exactly the hardware required for the multimodal guitar-AI system.

Therefore:

MB1940 $\neq$ cAiGED PCB

Instead:

MB1940 $\rightarrow$ reference

cAiGED $\rightarrow$ derived custom implementation

48. Existing engineering baseline documents

Several previous engineering documents/projects provide useful context but should not be confused with the cAiGED baseline.

The STM32-RF-USB-Gateway project demonstrates previous experience with STM32, USB, RF, controlled impedance, antenna matching and four-layer PCB design.

The USB-to-Gigabit-Ethernet project demonstrates experience with high-speed USB, Ethernet, multiple power rails, external PHY/network magnetics and high-speed PCB design.

These projects are supporting experience/reference material.

They are not the current cAiGED architecture.

49. Current project status

The project should currently be considered approximately at the transition between:

system architecture / reference-design study

and

custom hardware architecture development.

The important baseline has already been established:

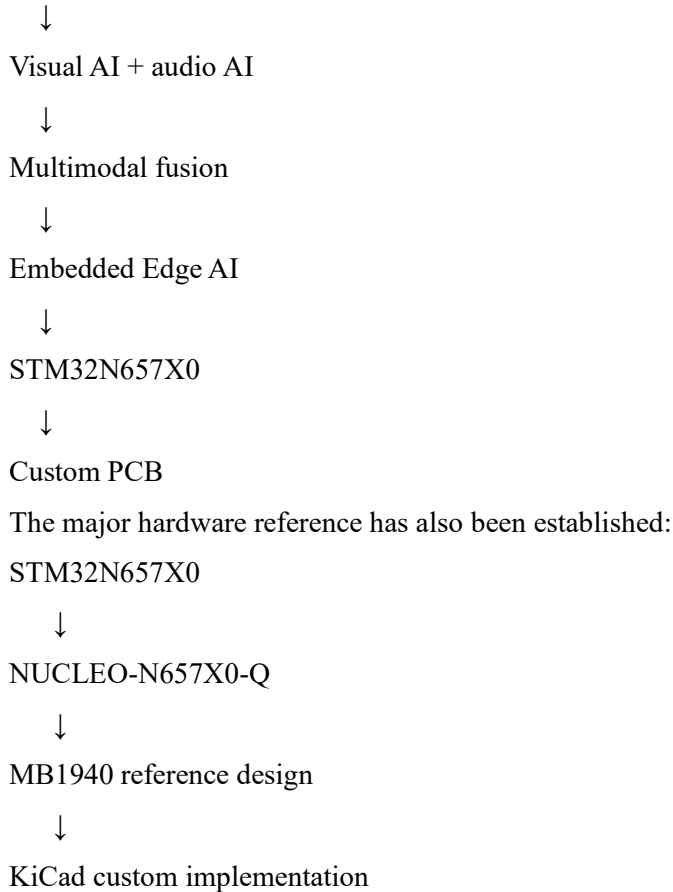
Application

↓

Multimodal guitar chord verification

↓

Camera + microphone



50. What is already considered decided

The following should be treated as established unless the user explicitly changes them:

Project concept

- Project name: cAiGED.
- Embedded Edge-AI guitar-learning system.
- Multimodal sensing.
- Camera + audio.
- CAGED-based initial chord scope.

MCU

- STM32N657X0 family.
- Current production device of interest: STM32N657X0H3Q.
- VFBGA264 package.

AI

- Neural-ART is an important reason for selecting the MCU.
- Visual and acoustic inference are conceptually separate.

- Multimodal fusion is part of the intended architecture.

Hardware development

- NUCLEO-N657X0-Q is the main development platform.
- MB1940 is the primary hardware reference.
- Custom PCB will be developed in KiCad.
- MB1940 is a reference, not a final schematic.

Interfaces/peripherals currently in scope

- Camera.
- External memory.
- External Flash.
- Audio/microphone.
- OLED/display.
- USB.
- Ethernet.
- microSD.
- Debug/programming.

51. Things that are NOT yet final

These must not be presented in future conversations as already decided.

AI:

- visual model architecture;
- audio model architecture;
- fusion architecture;
- training framework;
- quantization strategy;
- exact dataset;
- target accuracy;
- inference latency target.

Camera:

- sensor;
- resolution;
- FPS;
- lens;
- FOV;

- camera placement;
- connector;
- power;
- CSI-2 configuration.

Audio:

- microphone type;
- ADC/codec architecture;
- sample rate;
- bit depth;
- acoustic preprocessing;
- microphone placement;
- audio interface.

Memory:

- SDRAM vs PSRAM;
- exact RAM device;
- RAM density;
- Flash device;
- Flash density;
- bus width;
- memory clock;
- final memory map.

Display:

- OLED model;
- resolution;
- interface;
- size;
- refresh rate;
- graphics requirements.

Connectivity:

- USB connector;
- exact USB mode;
- Ethernet PHY;
- Ethernet connector;
- microSD implementation.

Power:

- exact power input;
- regulator topology;
- rail voltages;
- current budgets;
- sequencing.

PCB:

- layer count;
- stack-up;
- dimensions;
- manufacturer;
- impedance values;
- via technology;
- BGA escape strategy.

Firmware:

- RTOS;
- task architecture;
- boot architecture;
- update mechanism;
- storage format.

52. Highest-priority engineering questions

Before the custom schematic is considered frozen, the following questions should be answered.

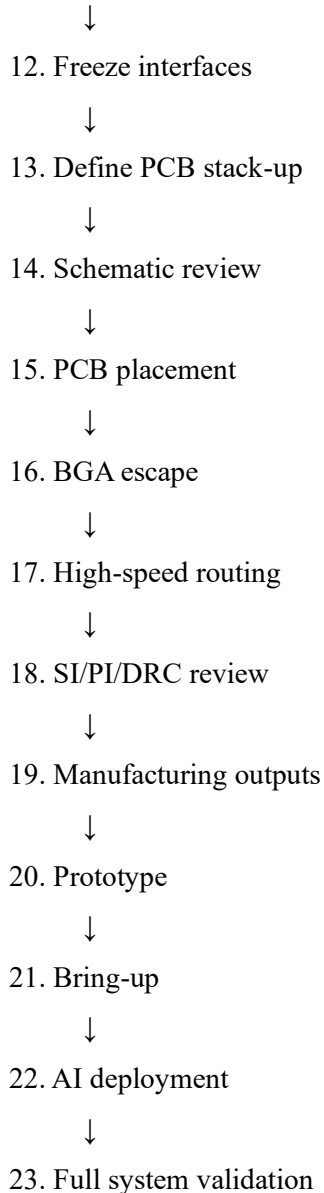
1. What exact camera sensor will be used?
2. What image resolution and frame rate are actually required?
3. What visual information does the AI need: full image, fretboard ROI, hand keypoints, finger positions, or some combination?
4. What microphone architecture will be used?
5. What sample rate and bandwidth are required?
6. What audio representation produces the best embedded classification performance?
7. What models can realistically run on Neural-ART?
8. How much RAM does the complete multimodal pipeline require?
9. How much external RAM bandwidth is required?
10. What external Flash density is required?

11. How should camera and audio events be synchronized?
 12. What inference latency qualifies as "real time" for the application?
 13. What exactly should the fusion system output?
 14. What level of diagnostic feedback should the OLED provide?
 15. Which MB1940 circuits are actually required?
 16. What should replace the Nucleo's ST-LINK circuitry?
 17. What are the exact cAiGED power requirements?
 18. What is the final PCB layer stack?
 19. What fabrication capabilities are required for the BGA and high-speed interfaces?
 20. What hardware test points are required?
-

53. Recommended engineering order from this point

The most rational order is:

1. Freeze system requirements
↓
2. Define camera requirements
↓
3. Define audio requirements
↓
4. Establish preliminary AI models
↓
5. Measure RAM / Flash / compute requirements
↓
6. Study MB1940 in detail
↓
7. Build KiCad STM32N6 reference schematic
↓
8. Verify every MB1940 section
↓
9. Define custom cAiGED peripherals
↓
10. Freeze memory architecture
↓
11. Freeze power architecture



The critical point is that steps 1–5 should inform steps 8–13.

The PCB should not be finalized before the AI and memory requirements are sufficiently understood.

54. Official external references

The most important current external reference is ST's STM32N657X0 product documentation. ST currently identifies the device as an active product in volume production and documents the Cortex-M55, 4.2 MB SRAM and Neural-ART architecture. ([STMicroelectronics](#))

[STM32N657X0 — STMicroelectronics](#)

The NUCLEO-N657X0-Q page is the main reference for the evaluation platform and provides the MB1940 design files, manufacturing files, BOM and schematic. ([STMicroelectronics](#))

[NUCLEO-N657X0-Q — STMicroelectronics](#)

The MB1940 schematic is the primary schematic-level hardware reference.

[MB1940-N657X0Q-C02 schematic — STMicroelectronics](#)

The STM32N6 datasheet is the primary electrical reference for the MCU itself. ([STMicroelectronics](#))

[STM32N6 datasheet — STMicroelectronics](#)

ST also provides STM32N6 AI/camera software examples, including support for NUCLEO-N657X0-Q and several camera modules. ([GitHub](#))

55. Important source hierarchy for future work

When making engineering decisions, use the following hierarchy:

1. Official STM32N6 datasheet/reference manual

For:

- electrical requirements;
- pin functionality;
- power;
- memory interfaces;
- clocking;
- absolute maximums;
- peripheral capabilities.

2. Official MB1940 design

For:

- practical STM32N6 implementation;
- component topology;
- power;
- memory;
- high-speed interfaces;
- layout/reference implementation.

3. Official ST application notes/design guidelines

For:

- PCB layout;
- signal integrity;

- power;
- interface implementation.

4. Peripheral manufacturer datasheets/reference designs

For:

- camera;
- microphone/audio;
- display;
- memory;
- Ethernet PHY;
- connectors.

5. Experimental measurements

For:

- actual AI performance;
- actual latency;
- actual power;
- acoustic performance;
- camera performance.

General internet sources should not override the above.

56. Critical caution for future ChatGPT work

Future conversations about cAiGED should not assume that every component mentioned in MB1940 belongs on the final cAiGED PCB.

The correct question is always:

"What function does this circuit perform, and does cAiGED actually require that function?"

Similarly, a component should not be selected merely because it appears on the Nucleo.

The reference design establishes a proven implementation, but the final architecture must be derived from the cAiGED requirements.

57. Compact architecture summary

For quick reference:

PROJECT

cAiGED

PURPOSE

Embedded AI guitar chord verification / learning system

INITIAL MUSICAL SCOPE

CAGED:

$C \rightarrow A \rightarrow G \rightarrow E \rightarrow D$

SENSORS

1. Camera
2. Microphone

AI

Visual model

+

Audio model

↓

Multimodal fusion

↓

Chord/performance decision

USER INTERFACE

OLED/display

MAIN MCU

STM32N657X0

Current production part:

STM32N657X0H3Q

MCU

Cortex-M55

Up to 800 MHz

4.2 MB SRAM

Neural-ART

600 GOPS

MIPI CSI-2

ISP

Ethernet

USB

External memory interfaces

JPEG/H.264 hardware

EXTERNAL MEMORY

External RAM — exact technology TBD

External Flash — exact device TBD

microSD — removable storage

CONNECTIVITY

USB

Gigabit Ethernet

microSD

DEVELOPMENT PLATFORM

NUCLEO-N657X0-Q

REFERENCE DESIGN

MB1940

CUSTOM PCB

KiCad

REFERENCE STRATEGY

MB1940 is the validated starting point,
not the final cAiGED schematic.

CURRENT DEVELOPMENT PHASE

System architecture /

AI feasibility /

MB1940 study /

KiCad hardware architecture

MAIN OPEN AREAS

Camera

Audio

AI models

Fusion

Memory

Power

PCB stack-up

Debug

Mechanical design

58. Handoff prompt for a new chat

The following can be pasted directly into a new ChatGPT conversation:

I am continuing development of my cAiGED project. Treat the following as the current project baseline.

cAiGED is a custom embedded Edge-AI guitar-learning/chord-verification system. The initial objective is to verify guitar chord playing using two independent sensing modalities: a camera observing the guitarist's fretboard/finger configuration and a microphone observing the resulting guitar sound.

The initial musical scope is the CAGED chord sequence:

$C \rightarrow A \rightarrow G \rightarrow E \rightarrow D$.

The conceptual architecture is:

Camera

→ visual preprocessing

→ visual AI

→ visual chord/fingering result

Microphone

→ audio acquisition/preprocessing

- audio AI
- acoustic chord result

Visual result + acoustic result

- multimodal fusion
- chord/performance decision
- OLED/display feedback.

The project is intended to operate locally as an embedded Edge-AI system rather than relying on a PC/cloud for inference.

The current MCU baseline is STM32N657X0, specifically the production STM32N657X0H3Q in VFBGA264. ST specifies a Cortex-M55 up to 800 MHz, 4.2 MB contiguous SRAM, external memory interfaces, MIPI CSI-2, ISP, Ethernet, USB, and the Neural-ART accelerator. The Neural-ART accelerator is one of the main reasons for selecting this MCU and is specified by ST at up to 600 GOPS.

The current hardware development strategy is NOT to design the STM32N6 circuit from scratch.

The primary hardware reference is STMicroelectronics' NUCLEO-N657X0-Q, whose main board design is MB1940.

ST provides:

- MB1940 Altium board design files
- MB1940 manufacturing files
- MB1940 BOM
- MB1940-N657X0Q-C02 schematic

MB1940 is the reference implementation, NOT the final cAiGED PCB.

The intended workflow is:

MB1940

- study every circuit

- understand why each circuit exists
- port/reconstruct relevant sections in KiCad
- verify against ST documentation
- remove Nucleo-specific circuitry
- retain/adapt the STM32N6 core architecture
- add cAiGED-specific peripherals
- design the custom PCB.

The final cAiGED PCB is expected to include, subject to final requirements:

- STM32N657X0H3Q
- external RAM
- external Flash
- camera interface
- microphone/audio acquisition
- OLED/display
- USB
- Gigabit Ethernet
- microSD
- debug/programming interface
- appropriate power architecture
- clocks/reset/decoupling
- supporting high-speed PCB infrastructure.

The exact camera, microphone/audio architecture, OLED, external RAM, external Flash, Ethernet PHY, USB implementation, power-tree, PCB layer stack-up, board dimensions and connector selection are NOT yet frozen.

The AI architecture is also not fully frozen.

The visual model could potentially perform direct chord classification or a more structured task such as fretboard/finger/keypoint detection followed by chord interpretation. The audio model could use waveform, spectrogram, mel-spectrogram, chroma or another representation. These are open engineering questions and should be determined experimentally.

The fusion architecture is not frozen either. Possible approaches include rule-based decision fusion, confidence-weighted fusion, feature-level fusion or neural late fusion. Do not assume one has already been selected.

The project should initially focus on constrained chord verification rather than arbitrary guitar transcription or general-purpose music recognition.

The important system-level problem is to determine whether the guitarist's physical fingering and the resulting acoustic chord agree with the expected target chord.

The project should be developed in stages:

1. Define system requirements.
2. Establish camera requirements.
3. Establish audio requirements.
4. Develop preliminary visual/audio AI models.
5. Measure inference, RAM, Flash and bandwidth requirements.
6. Study MB1940.
7. Reconstruct the STM32N6 reference architecture in KiCad.
8. Verify the reference implementation against ST documentation.
9. Define custom cAiGED peripherals.
10. Freeze memory and power architecture.
11. Design PCB stack-up.
12. Perform schematic review.
13. PCB placement/BGA escape.
14. High-speed routing.
15. SI/PI/DRC review.
16. Prototype.
17. Hardware bring-up.
18. AI deployment.
19. Full multimodal validation.

The PCB will be significantly more demanding than my previous STM32/USB/RF projects because it involves a 264-ball BGA, external high-speed memory, MIPI CSI-2, Ethernet, USB and high-performance MCU power/clock architecture.

Previous PCB experience that is relevant includes:

- STM32L432 designs
- USB Full-Speed
- RF/nRF24L01P
- 50-ohm RF routing
- USB differential routing
- four-layer PCBs
- Ethernet/USB 3.x design
- high-speed differential interfaces
- power distribution
- controlled impedance
- SI considerations
- production test systems.

However, those previous designs are supporting experience only and must NOT be treated as the cAiGED hardware baseline.

When helping me with cAiGED, act as a senior electronics/hardware engineer with expertise in:

- STM32N6
- Edge AI
- Neural-ART
- embedded systems
- camera interfaces
- audio acquisition
- high-speed PCB design
- BGA escape
- DDR/SDRAM/PSRAM
- XSPI
- MIPI CSI-2
- USB

- Ethernet
- signal integrity
- power integrity
- EMC/EMI
- KiCad
- embedded firmware
- AI deployment.

Be rigorous. Do not invent design decisions. Clearly distinguish:

1. decisions already established,
2. requirements that still need to be defined,
3. recommendations,
4. assumptions,
5. facts taken from ST documentation,
6. experimental results.

The most authoritative hardware references are:

- STM32N657X0 datasheet/reference manual
- official NUCLEO-N657X0-Q / MB1940 design
- official ST application notes/design guidelines
- datasheets/reference designs of the final selected peripherals.

Do not blindly copy MB1940. Analyze each circuit and determine whether cAiGED actually needs it.

The immediate hardware objective is to understand MB1940 deeply enough to create a clean KiCad cAiGED schematic architecture based on it.

That is the baseline I would carry into the next conversation. The most important distinction to preserve is that **cAiGED is a multimodal Edge-AI system, while MB1940 is only the hardware reference from which the custom cAiGED board is being derived.**